

Potruga za blagom

Ograničenje vremena: 8 s Ograničenje memorije: 256 MiB

Tražiš davno izgubljeno blago na ogromnom polju ćelija veličine $N \times N$ uz pomoć magičnog kompasa. Ukupno postoji K kovčega s blagom skrivenih u različitim ćelijama. Tvoj cilj je da pronadeš sve kovčege sa blagom!

Kada postaviš kompas na neku ćeliju, on će pronaći najkraći put do kovčega s blagom koristeći samo pomjeranja ulijevo, nagore, udesno i nadolje (tj. do najbližeg kovčega prema Menhetn rastojanju). Zatim će pokazati smjer prvog koraka. Ako postoji više ispravnih prvih koraka koji vode do najbližeg kovčega (ili više njih), kompas će vratiti sve te smjerove. Ako ćelija sadrži blago, kompas će to naznačiti.

Kad god pronadeš kovčeg s blagom, možeš isprazniti njegov sadržaj, ali ne možeš ukloniti kovčeg pošto je pretežak. Tvoj kompas ne zna da li je kovčeg pun ili prazan pa će pokazivati put ka najbližem kovčegu, bilo da je prazan ili pun.

Pokušaj da pronadeš lokacije svih kovčega s blagom koristeći što manje upita kompasu.

Zadatak

Ovo je interaktivni zadatak. U svakom test slučaju (tj. pri svakom izvršavanju tvog programa), program će morati da riješi nekoliko potraga na blago. Sa sistemom za ocjenjivanje treba da komuniciraš pomoću biblioteke koju su obezbijedili organizatori. Biblioteka sadrži sljedeće deklaracije:

- **void** *NextHunt*(**int** & N , **int** & K) — pozovi ovu funkciju da započneš sljedeću potragu za blagom. Ona će postaviti parametar N na veličinu mreže a parametar K na broj kovčega blaga. Ako nema više potraga na blago koje treba riješiti u trenutnom izvršavanju, funkcija će postaviti N i K na -1 ; u tom slučaju treba da završiš program sa izlaznim kodom 0. Napomena: ovu funkciju možeš pozvati i prije nego što pronadeš sva blaga u trenutnoj potrazi, npr. ako pokušavaš da osvojiš samo djelimičan broj poena.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — ovo su konstante koje se koriste u povratnim vrijednostima funkcije *Query* (vidi ispod).
- **int** *Query*(**int** x , **int** y) — ako ćelija na koordinatama (x, y) sadrži blago, ova funkcija vraća *TREASURE*. U suprotnom vraća zbir jedne ili više konstanti *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* i *DIR_DOWN*, označavajući koja pomjeranja kompas vraća kada se postavi na ćeliju (x, y) . Koordinate x i y moraju biti cijeli brojevi od 0 do $N - 1$. (Napomena: u ovom zadatku y -koordinate rastu od vrha ka dnu.)

Nakon što *NextHunt* vrati $N = K = -1$, tvoj program više ne smije pozivati ni *NextHunt* ni *Query*. Takođe ne smije pozvati *Query* prije prvog poziva funkcije *NextHunt*. Ako program ne ispoštuje ova ograničenja, ili ako napravi više od 1000 upita tokom jedne potrage za blagom, ili ako funkciji *Query* proslijedi vrijednosti x i/ili y van dozvoljenog opsega, biblioteka će prekinuti program i vratiti run-time-error (RTE) za trenutni test slučaj.

Blago se smatra pronađenim ako si makar jednom postavio upit za ćeliju koja ga sadrži. Ako program pozove *NextHunt* prije nego što pronađe sva blaga iz trenutne pretrage, to se ne smatra greškom, ali će uticati na rezultat (više o bodovanju u nastavku).

Za pristup biblioteci, program treba da uključi heder fajl `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Heder fajl možeš preuzeti ovdje: `treasurehuntlib.h`.

Kako bi ti pomogli u razvoju rješenja, dostupna je jednostavna implementacija biblioteke ovdje: `treasurehuntlib-public.cpp`. Da bi je kompajlirao zajedno sa svojim programom, jednostavno dodaj njeno ime kao parametar kompajleru, na primjer:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

ako je `foo.cpp` ime fajla koji sadrži tvoje rješenje.

Implementacija fajla `treasurehuntlib-public.cpp` podržava, dodatno u odnosu na funkcije odozgo, i funkciju `void InitFromFile(const char *fileName)`, koja čita listu potraga za blagom iz fajla i omogućava vam da se igrate sa njima umjesto da se lista pretraga na blago generiše random. Za više detalja o ovoj funkciji pogledaj fajl `treasurehuntlib-public.cpp`.

Na evaluacionom serveru koristiće se drugačija implementacija biblioteke, tako da ne treba da praviš nikakve pretpostavke o tome kako je tačno implementirana biblioteka. Ipak, možeš pretpostaviti da biblioteka ne zagađuje globalni prostor imena nikakvim deklaracijama osim navedenih (*NextHunt*, *Query* i pet konstanti).

Tvoj program ne smije čitati sa standardnog ulaza niti pisati na standardni izlaz, jer će ih naša implementacija biblioteke na evaluacionom serveru koristiti za komunikaciju sa ostatkom evaluacionog okruženja.

Ulaz

Ograničenja

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Tokom jednog izvršavanja programa biće najviše 100 000 potraga za blagom.
- Tokom jedne potrage za blagom možeš napraviti najviše 1000 upita.

Podzadaci

- Podzadatak 1 (10 poena) $K = 1$
- Podzadatak 2 (30 poena) $K = 2$
- Podzadatak 3 (60 poena) $K = 3$.

Bodovanje

Jedan podzadatak može se sastojati od nekoliko test slučajeva (nekoliko izvršavanja tvog programa), a svaki test slučaj može sadržati nekoliko potraga za blagom. Za potrebe bodovanja, sve potrage za blagom datog podzadatka ocjenjuju se zajedno, bez obzira na to kako su prvobitno bile raspoređene među test slučajevima. Za i -tu potragu za blagom veličinu mreže označavamo sa $N_i \times N_i$, broj kovčega blaga sa K_i , broj upita koje je napravio program sa Q_i , i broj pronađenih blaga sa F_i . Neka takođe S označava ukupan broj poena dostupan za taj podzadatak. Tada je broj poena dodijeljen rješenju za taj podzadatak:

- Ako je program uvijek pronašao sve kovčege sa blagom (tj. ako je $F_i = K_i$ za sva i), rezultat zavisi od $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ poena, gdje je } f(t) = \begin{cases} 1, & t \leq 11 \\ 1 - (t - 11)/9, & 11 \leq t \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Ako program nije uvijek pronašao sve kovčege sa blagom (tj. postoji i takav da je $F_i < K_i$), dobija

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ poena.}$$

Drugim riječima, dobijaš polovinu poena za pronalaženje svih kovčega sa blagom, a drugu polovinu za njihovo pronalaženje sa što manje upita. Za maksimalan broj poena, tvoje rješenje treba da pronade sva blaga u najviše $11 \lceil \log_2 N_i \rceil$ upita. Između $11 \lceil \log_2 N_i \rceil$ i $20 \lceil \log_2 N_i \rceil$ upita rezultat opada linearno; a ako tvoje rješenje napravi više od $20 \lceil \log_2 N_i \rceil$ upita, dobiće samo prvu polovinu poena za pronalaženje svih blaga. (Simboli $\lceil \cdot \rceil$ znače da se vrijednost $\log_2 N_i$ zaokružuje naviše na najbliži cijeli broj.)

Ako gornje formule daju necijeli broj poena za podzadatak, rezultat će biti zaokružen na najbliži cijeli broj.

Ako program ima grešku tokom izvršavanja ili se ne pridržava prethodno opisanog protokola za korišćenje biblioteke, dobiće 0 poena za cio podzadatak. Da bi dobio djelimične poene za pronalaženje samo podskupa kovčega sa blagom, program mora pravilno završiti pretragu pozivom funkcije *NextHunt*.

Primjer

Poziv	Povratna vrijednost
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$